CHALLENGE SHEET

Aegis Ctf

Chained API trust vulnerabilities across REST, GraphQL, and gRPC.

DIFFICULTY	TRACK	HOST PORT	LAB ID
HARD	WEB	3000, 50051	aegis-ctf

EASY | MEDIUM | HARD | INSANE

VERIFY THE SOLVE

```
python3 ../scripts/check.py aegis-ctf
```

Brief

AEGIS is an internal employee portal for a fictional company. From the outside, it looks like a simple web application with a login page, employee profiles, and self-service features.

Behind the application, multiple backend services communicate with the same underlying data. Your goal is to investigate how these services interact and identify the trust weaknesses between them.

Goal

Recover the full flag by progressing through the different API services exposed by the application.

The challenge involves multiple API technologies, including REST, GraphQL, and gRPC.

See how to play for the usual solve loop.

Difficulty

Hard

Experience with Burp Suite and modifying HTTP requests is recommended.

Prerequisites

- Docker Desktop or Docker Engine with Docker Compose
- A web browser
- Burp Suite or another HTTP intercepting proxy
- Basic knowledge of REST APIs, GraphQL, and gRPC

Setup

From the lab directory, run:

```
docker compose up --build
```

Wait for the application to start.

Connection Information

The challenge is available at:

```
http://localhost:3000
```

This is the main entry point to the challenge.

Starting Access

A low-privilege employee account can be discovered from the application. Carefully inspect the starting page and the requests it makes.

Do not rely on generic credential guessing.

Reset

The challenge state is stored in memory. To reset the challenge, restart the container:

```
docker compose restart
```

Flag Format

```
duck\{...\}
```

The complete flag requires progress through multiple parts of the application.

Rules

- Only interact with the services exposed by this local challenge.
- Do not attack external systems.
- Generic credential brute-forcing is not part of the intended solution.
- Automated enumeration and fuzzing are allowed where appropriate.
- Carefully inspect HTTP responses and API behavior for additional information.

Good luck!